TDA352/DIT352: Cryptography LP2, Fall 2026

Lecture Notes on

Course Introduction, Perfect Security, OTP

Lecturer: Elena Pagnin Lecture 1 on Nov 3

Scribes: Lucia Lavagnino, Kelsey Melissaris & Emil Babayev Last update: November 3, 2026

Contents

1 Introduction to Cryptography **1**
1.1 A Very Brief and Abridged History 1
1.2 Kerckhoffs' Principle 2
1.3 Modern Cryptography 2

2 Symmetric Encryption **3**
2.1 Secure Communication 3
2.2 Symmetric Encryption 4
2.3 Perfect Secrecy 5
2.4 The One Time Pad (OTP) 6
2.5 Shannon's Theorem(s) 6

1 Introduction to Cryptography

Etymologically, the word "cryptography" derives from the Ancient Greek words *kryptós*, meaning 'hidden, secret', and *graphein*, meaning 'to write'; cryptography refers to the practice and study of techniques for secure communication. Sociologically, the need for secrecy originates in fundamental human desires, e.g., protect resources, or gain advantage in war. As such, cryptography has woven itself into the fabric of human history, albeit with a somewhat understated presence.

The mission of cryptography has evolved, and that of cryptographers has converged to a single overarching goal: *to make our digital world safe*. Modern cryptography models a wide array of security properties, including: confidentiality; data integrity; authenticity; entity identification; access control or authorisation; anonymity; non-repudiation; and privacy. Security is a critical consideration in system design and development; professionals across various fields——system architects, software developers, and security specialists—— must develop a robust understanding of these security principles.

In this course we will explore how cryptographic primitives are meticulously modeled to formalise our intuitive understanding of security, how cryptographic protocols are rigorously proven to satisfy these security notions in the presence of malicious actors, and how critical vulnerabilities can be identified in flawed designs.

1.1 A Very Brief and Abridged History

20th Century, BC: Egyptian Monks. The earliest known instances of secret encodings can be traced back to Egyptian monks, who devised a photocryptographic method that used non-standard hieroglyphics to conceal communication from outsiders.

100 BC: Julius Caesar. The Caesar cipher—named after the Roman general and statesman Gaius Julius Caesar—is among the earliest encryption schemes. As a substitution cipher the scheme is simple: each letter in the plaintext (message) is replaced by the letter a fixed number of positions away in the alphabet. In Caesar's case a shift of three was commonly used: 'A' is replaced by 'D', 'B' by 'E', and so on. The Caesar cipher is insecure by our modern standards, being vulnerable to both frequency analysis and brute-force attacks.[1]

1587 AD: Queen Mary. Mary Queen of Scots [Wik25d] sought to orchestrate the assassination of her cousin, Queen Elizabeth I, to claim the throne. As part of her intricate scheme she dispatched an encoded message to Sir Anthony Babington. The message featured unusual symbols and similarly used a substitution cipher in which each symbol represented a letter. Before long the guards realised that the most common symbol in the message represented to the most common letter in the English language, thereby mounting what is

[1] A brute-force attack simply tries every possible key—in this case trying every possible shift—and checking if the resulting message makes sense. A frequency analysis does the same but informs its choices with the frequency of letters in a language and the frequency of symbols in a ciphertext, starting by replacing the most frequent symbol with the most frequent letter.

referred to as a frequency analysis attack. Consequently, Mary's 'secret' communication was decrypted and became the proof of her treason. Mary was executed.

1941: World War II. In 1941 Mavis Batey worked to decipher an 'incomprehensible' stream of characters produced by Germans via the Enigma Machine [Wik25a], a then state-of-the-art mechanism used to encrypt and decrypt messages. One day she intercepted an encryption that contained no occurences of the letter 'L' and determined that the probability a long plaintext (message) similarly contained no occurences of the letter 'L' was implausibly low. She conjectured that the ciphertext must have been an encryption of a message consisting of *only* 'L' (likely typed to test the machine). If her conjecture was true, it implied that the Enigma Machine would never encrypt a letter as itself. This arguably very small bias was sufficient for Mavis' team to gain an advantage in deciphering subsequent intercepted messages, including one detailing a planned attack on the UK by Italy during the Battle of Cape Matapan. The decrypted message provided invaluable information that contributed to defeating the enemy.

1920-2015: Gene Grabeel. Gene Grabeel [Wik25b] worked to decipher intercepted Soviet ciphertexts. During World War II Russian militaries used the *one time pad* (OTP, see Definition 4 below), an encryption scheme that is perfectly secure as long as the encryption key is used only one time. Increasing pressure due to the German advance on Moscow, resulted in the neglect of this fact. Soviet forces started to re-use encryption keys, thereby undermining the security guarantees of the OTP. This re-use was discovered and exploited by Gene's team to decrypt a large share of secret communications, aiding the Allies in their victory.

Historically, many cryptographic failures can be attributed to three fundamental issues: a lack of rigorous security estimates/analyses, a reliance on security through obscurity, and a tendency toward bad security hygiene.

To read more about the use of cryptography throughout history click here [Wik25c].

1.2 Kerckhoffs' Principle

Auguste Kerckhoffs (19 January 1835 – 9 August 1903), a Dutch linguist and cryptographer, surveyed military cryptography in his paper *La Cryptographie Militaire* [Ker83] and, in that work, both made a plea for considerable improvement and issued practical advice. His work introduced the following six principles of practical cipher design:

1. The system should be, if not theoretically unbreakable, unbreakable in practice.
2. The design of a system should not require secrecy, and compromise of the system should not inconvenience the correspondents (now called Kerckhoffs's principle, below).
3. The key should be memorable without notes and should be easily changed.
4. The cryptograms should be transmittable by telegraph.
5. The apparatus or documents should be portable and operable by a single person.
6. The system should be easy, neither requiring knowledge of a long list of rules nor involving mental strain.

Of these six principles, one became a cornerstone of modern cryptography, known today as Kerckhoffs' Principle. This principle serves as one of the foundations of modern cryptographic design. In essence, the security of a system should not depend on an attacker's ignorance of how the system operates, but rather on the secrecy of the key. In the words of Claude Shannon: The enemy knows the system; therefore, we must design systems under the assumption that the enemy will immediately gain full familiarity with them. In other words, there should be *no security through obscurity.*

Kerckhoffs' Principle (1884): A cryptosystem should be secure even if everything about the system apart from the key is public knowledge.

1.3 Modern Cryptography

In modern cryptography (from the 1980s onward) one first establishes rigorous definitions, answering: What does "secure" really mean? What are our attacker's goals and resources? What makes a system "hard" to attack? One then uses formal methods to logically argue security. In contrast, cryptanalysis relies on heuristics to determine exact security levels, and requires practical solutions. Together, cryptography and cryptanalysis form the broader field of cryptology. In this course, we focus on cryptography and, as such, are primarily concerned with answering the question:

The following tools are useful in finding an answer.

Formal Definitions. We provide formal mathematical definitions of security, e.g., frameworks such as game-based security that quantifies the probability of an adversary's success in breaking a system relative to an algorithmic definition of security (called a "game").

Computational Assumptions. We formalise precise mathematical assumptions about the computational intractability of certain problems, formalising statements such as "the discrete logarithm problem is hard."

Proofs of Security. We introduce frameworks and techniques for analysing the security of cryptographic constructions, including proofs by reduction. In such proofs, one shows that breaking a scheme would contradict a well-established mathematical assumption. Therefore, assuming the validity of the underlying assumption, the scheme is considered secure.



Figure 1: Roughly, in a proof of security by reduction a successful adversary $\mathcal{A}$ against some cryptographic primitive can be used to construct an adversary $\mathcal{A}'$ that solves an instance of some computational problem given as input by the challenger $\mathcal{C}$; under the assumption that the computational problem is hard the adversary $\mathcal{A}'$ does not exist, and therefore the adversary $\mathcal{A}$ against the cryptographic primitive cannot exist.

2 Symmetric Encryption

Cryptographers are paranoid, so their guiding principle is straightforward: trust as little as possible. This mindset naturally lends itself to the anticipation of worst-case scenarios, and the design of systems that ensure security in the worst case—even in the presence of adversaries. Cryptographic primitives and protocols are constructed to achieve specific security goals, even when attempts are made to subvert their intended functionality. As an example, we illustrate this process by presenting the problem of secure communication, translating our intuition into a precise security definition, presenting a method for secure communication, and proving that this construction satisfies our definition.

2.1 Secure Communication

We begin by introducing the cast of characters commonly used to describe cryptographic tasks, a convention first introduced by Ron Rivest et al. (in "Methods for Obtaining Digital Signatures and Public-Key Cryptosystems", [RSA78]) and widely adopted by the cryptographic community: **Alice**, **Bob**, and **Eve**. Alice acts as the sender—in this scenario, she wishes to transmit a secret message to Bob. Bob, in turn, is the receiver. Eve represents the adversary: an eavesdropper attempting to intercept the communication between Alice and Bob and uncover the exchanged secrets [2].

[2]The use of "familiar" names allows for inclusive language in technical discussions and lowers the barrier for relating to the scenarios being presented. Additionally, the simplicity of the names makes it easier for beginners to grasp cryptographic concepts without getting bogged down by technical jargon.

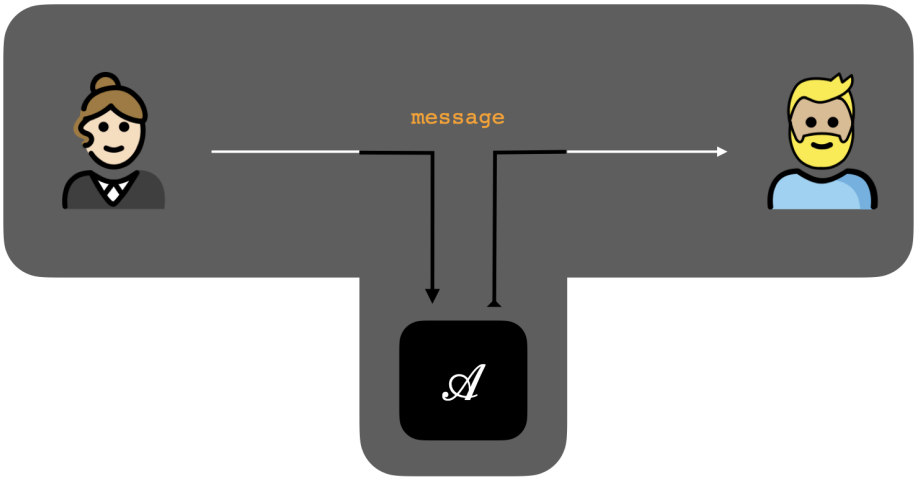


Figure 2: Insecure communication in the presence of an eavesdropper: Alice attempts to send a message to Bob which is intercepted and forwarded by our adversary $\mathcal{A}$, called Eve.

As noted, many historical challenges in cryptography arose from a failure to clearly define or understand the goals to be achieved. To establish secure communication over an insecure channel, we must first specify what we mean by "security." This requires defining the adversary's objectives, capabilities, and limitations. For now, we can summarise the scenario as follows:

Adversarial Goal: To learn information about the message sent from Alice to Bob.

Adversarial Capacity: The adversary (Eve) can eavesdrop on the channel and observe all messages exchanged.

Adversarial Limitations: We give no limitations on the tools Eve can use.

2.2 Symmetric Encryption

A symmetric cryptosystem is a mechanism with which Alice and Bob can communicate securely[3]. We begin by specifying syntax in Definition 1—that is, describing the intended functionality of a symmetric encryption scheme—and then introduce a corresponding notion of security.

Definition 1 (**Symmetric Encryption Scheme**)**.** A symmetric encryption scheme parameterised by the key space $\mathcal{K}$, message space $\mathcal{M}$, and ciphertext space $\mathcal{C}$, is described by a triple of Probabilistic Polynomial Time (PPT)[a] algorithms $(\texttt{KeyGen}, \texttt{E}, \texttt{D})$ with the following syntax:

$\texttt{KeyGen}(1^n) \to k$**.** Key generation is a randomised algorithm that takes as input a security parameter $n \in \mathbb{N}$ (input as a string of 1's of length n) and outputs a shared key $k \in \mathcal{K}$. This algorithm is often implicit and assumed to be $k \xleftarrow{\$} \mathcal{K}$.[b]

$\texttt{E}(k, m) \to c$**.** Encryption is a (possibly randomised) algorithm that takes as input a key $k \in \mathcal{K}$ and a message $m \in \mathcal{M}$, and outputs a ciphertext $c \in \mathcal{C}$.

$\texttt{D}(k, c) \to m$**.** Decryption is a **deterministic** algorithm that takes as input a key $k \in \mathcal{K}$ and a ciphertext $c \in \mathcal{C}$, and outputs a message $m \in \mathcal{M}$.

[a] A PPT algorithm is a randomised algorithm whose running time is bounded by a polynomial in the length of its input or a security parameter n (c.f. Definition 23.4 in Pass & Shelat).

[b] The notation $x \xleftarrow{\$} \mathcal{X}$ denotes uniformly sampling an element x from the set $\mathcal{X}$.

For simplicity we require *perfect correctness*, meaning that for any key generated with `KeyGen` decryption always works as intended. This is formalised in Definition 2.

[3] Symmetric encryption is also ofte called private-key encryption or symmetric-key encryption

Definition 2 (Correctness). A symmetric encryption scheme $(\mathtt{KeyGen}, \mathtt{E}, \mathtt{D})$ is **perfectly correct** if $\forall m \in \mathcal{M}$:

$$\mathsf{Pr}\left[\mathtt{D}(k, \mathtt{E}(k, m))\right] = 1.$$

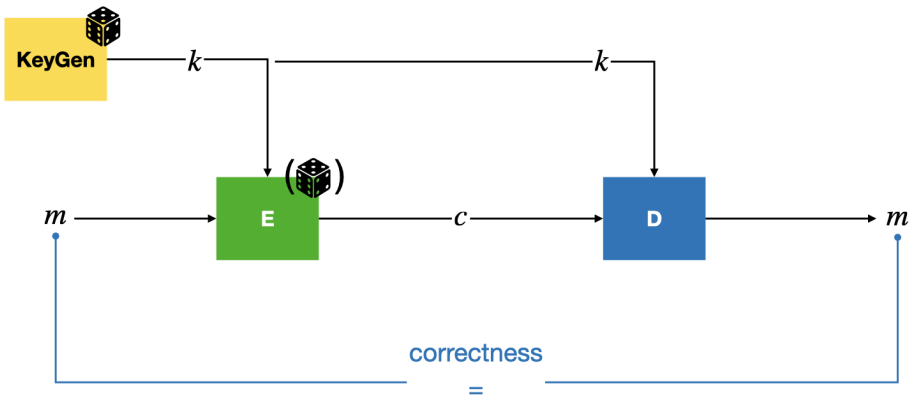


Figure 3: A visualisation of Symmetric Encryption: the algorithm `KeyGen` (yellow) outputs a key k, the algorithm `E` (green) takes as input the key and a message m and outputs a ciphertext c, the algorithm `D` takes as input the key and a ciphertext c and outputs the message m; correctness requires that the messages are equal.

2.3 Perfect Secrecy

Now that we have presented symmetric encryption (Definition 1), we must answer the following:

What does it mean for a symmetric encryption scheme to be secure?

Intuitively, we simply want that the adversary should not learn the message. Formally, the key insight of Shannon was that in a secure encryption scheme a ciphertext should not reveal *any* information about the message (apart from the length)[4]. In Definition 3 we present the notion of *perfect secrecy*[5], which encapsulates this insight.

Definition 3 (Perfect Secrecy). A symmetric encryption scheme $(\mathtt{KeyGen}, \mathtt{E}, \mathtt{D})$ is **perfectly secret** if for all pairs of messages $m_0, m_1 \in \mathcal{M}$ and for all ciphertexts $c \in \mathcal{C}$ it holds that:

$$\mathsf{Pr}\left[c \leftarrow \mathtt{E}(k, m_0) \mid k \leftarrow \mathtt{KeyGen}(1^n)\right] = \mathsf{Pr}\left[c \leftarrow \mathtt{E}(k, m_1) \mid k \leftarrow \mathtt{KeyGen}(1^n)\right].$$

The definition essentially states that for every pair of messages, the probability that a ciphertext is an encryption of the messages must be equal. In other words, all messages are equally likely to be the decryption of any ciphertext, and therefore Eve does not learn anything about a message given a ciphertext.

Perfect secrecy is a form of **information-theoretic** (also known as unconditional) security notion; this security property holds no matter the computational power and resources (including time-, or space-limit) of the adversary.

Computational security, in contrast, is a property that depends upon on the conjectured gap between the complexity classes P and NP: legitimate users run efficient algorithms (for example, `E` and `D`), but the adversary can only attack the scheme by performing an inefficient computation (for example, exhaustively searching the key space $\mathcal{K}$ for the key). The P versus NP problem is a major unsolved problem in theoretical computer science that, informally, asks if every problem whose solution can be quickly verified can also be quickly solved.

The desired computational gap exists for "hard" problems (those with an efficient algorithm to verify a solution to the problem, but no efficient algorithm is known to find a solution). Exactly the set of problems that are considered to be computationally hard for our adversary depend upon what we believe to be our adversary's resources (classical vs quantum, storage space, time). Computational solutions are usually more practical than information-theoretic solutions, but these solutions always come with a "best before date."

[4] Leaking the length of the message, or an upper bound on the length, is unavoidable.

[5] Perfect secrecy is also often called perfect security or information-theoretic security or unconditional security.

2.4 The One Time Pad (OTP)

An example symmetric encryption scheme is the One-Time Pad (OTP), which can be found in Definition 4. In this case the parameters are such that the key space $\mathcal{K}$ is equal to the message space $\mathcal{M}$—bitstrings with length the security parameter $n \in \mathbb{N}$: $\mathcal{K} = \mathcal{M} = \mathcal{C} = \{0,1\}^n$.

Definition 4 (One Time Pad Symmetric Encryption Scheme). The One Time Pad (OTP) for security parameter $n \in \mathbb{N}$ is described the triple of algorithms below:

$\texttt{KeyGen}(1^n) \to k$. Output $k \xleftarrow{\$} \{0,1\}^n$.

$\texttt{E}(k, m) \to c$. Output $c := k \oplus m$.

$\texttt{D}(k, c) \to m$. Output $m := k \oplus c$.

Encryption and decryption are simple. To encrypt a message $m \in \mathcal{M}$ with a key $k \in \mathcal{K}$ we simply output $m \oplus k$ where $\oplus$ is the bitwise XOR operation. Decryption works in the same way; to decrypt a ciphertext $c \in \mathcal{C}$ with the key $k \in \mathcal{K}$ we simply output $c \oplus k$.

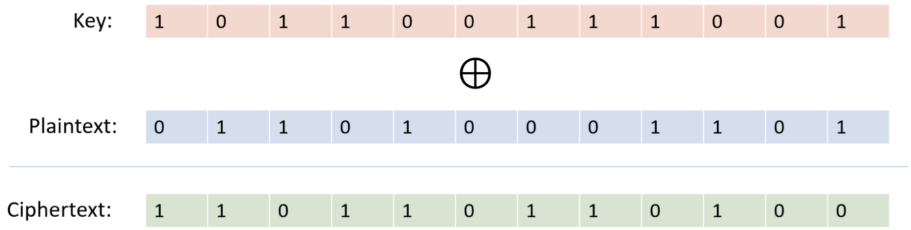


Figure 4: The One Time Pad Symmetric Encryption Scheme. The encryption operation is the bitwise XOR of the key $\texttt{101100111001} = k \in \mathcal{K}$ and the plaintext (message) $\texttt{011010001101} = m \in \mathcal{M}$. The output is ciphertext $\texttt{101100111001} \oplus \texttt{011010001101} = \texttt{110110110100} = c \in \mathcal{C}$. Recall that XOR is the exclusive OR of two bits: for $\texttt{b}, \texttt{b}' \in \{0,1\}$ the exclusive OR is $\texttt{b} \oplus \texttt{b}' = 1$ iff $\texttt{b} \neq \texttt{b}'$.

2.5 Shannon's Theorem(s)

We syntactically described a symmetric encryption scheme (Definition 1), presented the properties of correctness (Definition 4) and perfect secrecy (Definition 3), then presented a candidate symmetric encryption scheme: OTP (Definition 4). The next step is, as cryptographers, to mathematically prove that the OTP satisfies our definitions! The following is due to Shannon [Sha49].

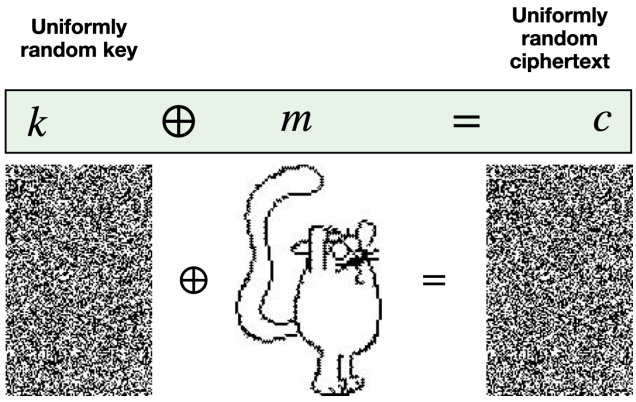


Figure 5: A visual representation of the perfect secrecy of the OTP. If the key k is uniformly random (in the space of $w \times h$ black and white pixels) then the ciphertext $c = k \oplus m$ is also uniformly random (in the space of $w \times h$ black and white pixels).

Theorem 1 (Shannon, 1949)**.** The One-Time Pad is a perfectly secure symmetric encryption scheme.

Proof. First, we show that OTP is a symmetric encryption scheme. This suffices to prove both correctness and perfect secrecy.

OTP is symmetric encryption. As in Definition 4, define the key space $\mathcal{K}$, message space $\mathcal{M}$ and ciphertext space $\mathcal{C}$ as $\mathcal{K} = \mathcal{M} = \mathcal{C} = \{0,1\}^n$, and the algorithms $\texttt{KeyGen}(1^n) : k \xleftarrow{\$} \{0,1\}^n$, $\texttt{E}(k,m) = k \oplus m$ and $\texttt{D}(k,c) = k \oplus c$.

Correctness (Definition 2) holds: for all $k \in \mathcal{K}$, and all $m \in \mathcal{M}$

$$\texttt{D}(k, \texttt{E}(k,m)) = k \oplus c = k \oplus (k \oplus m) = m.$$

For **perfect secrecy**, we notice that for every message $m \in \mathcal{M}$ and ciphertext $c \in \mathcal{C}$ there is exactly *one* key $k \in \{0,1\}^n$ such that $c = \texttt{E}(k,m)$: the key is $k = m \oplus c$. Thus, we have that:

$$\mathsf{Pr}\left[c \leftarrow \texttt{E}(k,m) \mid k \leftarrow \texttt{KeyGen}(1^n)\right] = \frac{1}{|\mathcal{K}|}.$$

Since this holds for every message m, we have that for all $m_0, m_1 \in \mathcal{M}$:

$$\mathsf{Pr}\left[\texttt{E}(k,m_0) \to c \mid k \leftarrow \texttt{KeyGen}(1^n)\right] = \frac{1}{|\mathcal{K}|} = \mathsf{Pr}\left[\texttt{E}(k,m_1) \to c \mid k \leftarrow \texttt{KeyGen}(1^n)\right].$$

□

The One-Time Pad has several drawbacks, such as key length, non-reusability, and malleability.

Key Length. The key is as long as the message.

Non-reusability. The key should only be used only to encrypt a single message.

Assume that the same key is used twice to produce $c_0 = k \oplus m_0$ and $c_0 = k \oplus m_0$. Assume the adversary $\mathcal{A}$ gets hold of the two ciphertexts, then $\mathcal{A}$ can compute:

$$c_0 \oplus c_1 = (k \oplus m_0) \oplus (k \oplus m_1) = m_0 \oplus m_1.$$

But $m_0 \oplus m_1$ conveys a lot of information about m_0 and m_1. This is not acceptable, see for example Figure 6.

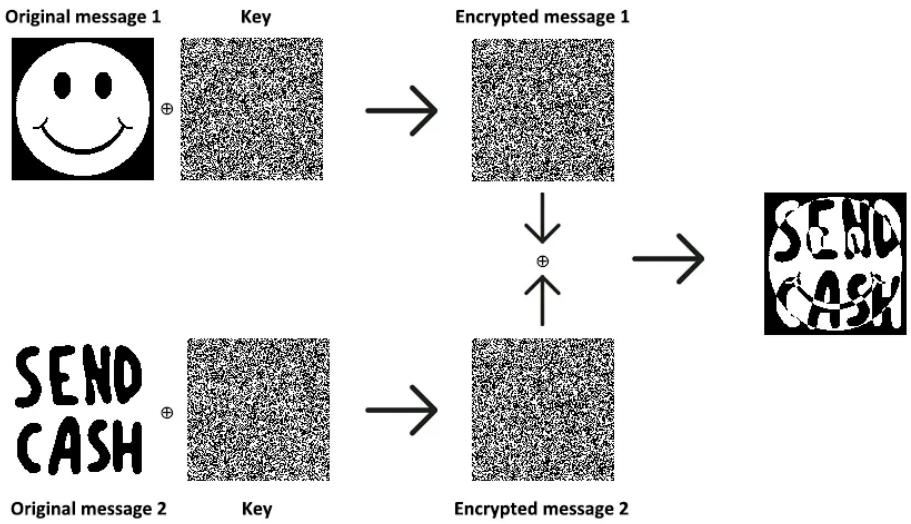


Figure 6: A visual representation of the insecurity of the OTP when a key is reused: the same key k is used to encrypt both the smile (Original message 1), resulting in the top ciphertext (Encrypted message 1), and "send cash" (Original message 2), resulting in the bottom ciphertext (Encrypted message 2)—the XOR of the two ciphertexts gives us a combination of the two messages. If we use the same key to encrypt two messages, a function of the messages can be revealed from just the ciphertexts.

Malleability. The ciphertext is (intentionally!) malleable, see Figure 7.



Figure 7: Malleability of OTP ciphertexts: The adversary $\mathcal{A}$ receives a ciphertext $c := \mathtt{E}(k,m)$ and, without any additional inputs, can compute a ciphertext $c^* = c \oplus e$ for some adversarially-chosen error $e \in \mathcal{M}$ such that $\mathtt{D}(k,c^*) = m \oplus e$. The adversary can compute known functions over the message knowing just the ciphertext.

We demonstrated the one-timeness (non-reusability) of the OTP by showing that patterns can be noticed from the ciphertext when a key is reused (Figure 6) and malleability by demonstrating that an adversary can compute known functions over a message with no information (Figure 7). We now prove that the first limitation of the OTP, key length, is unavoidable for any perfectly secure encryption scheme.

Theorem 2 (Shannon, 1940s)**.** If a symmetric encryption scheme $(\mathtt{KeyGen}, \mathtt{E}, \mathtt{D})$ with parameters $\mathcal{K}, \mathcal{M}, \mathcal{C}$ has perfect security then $|\mathcal{K}| \geq |\mathcal{M}|$.

Proof. Fix arbitrary $m_0 \in \mathcal{M}$ and $k_0 \in \mathcal{K}$, and let $c_0 = \mathtt{E}(k_0, m_0)$. Since the encryption scheme has perfect secrecy, it holds that "every message m has the same probability to be encrypted to c":

$$\mathsf{Pr}[c_0 = \mathtt{E}(k,m)] = \mathsf{Pr}[c_0 = \mathtt{E}(k,m_0)] > 0.$$

Thus for each $m_i \in \mathcal{M}$ there is a key $k_i \in \mathcal{K}$ such that $E(k_i, m_i) = c_0$. But recall that each message must have a different key! If not, the same key k maps two plaintexts m_0 and m_1 to the same ciphertext $c_0 = \mathtt{E}(k,m_1) = \mathtt{E}(k,m_0)$, and correctness does not hold (the decryption of c_0 for the key k becomes ambiguous). Thus, there must be at least as many keys as messages, i.e., $|\mathcal{K}| \geq |\mathcal{M}|$.

□

Note. Perfect security is impractical. Knowing this, it is natural to ask how close to perfect security we can get while remaining practical? The solution is given by randomness. At the core of cryptography is randomness; the perfect secrecy of OTP comes from the use of one random key to mask/hide one message. Since we cannot reuse the key (otherwise we lose security), we ask: can we somehow 'expand' a key to get many new keys? Stay tuned!

Note. Notes on PRGs available in Lecture 2.

References

[Ker83] Auguste Kerckhoffs. "La cryptographie militaire". In: *Journal des Sciences Militaires* IX (1883), pp. 5–83.

[RSA78] Ronald L Rivest, Adi Shamir, and Leonard Adleman. "A method for obtaining digital signatures and public-key cryptosystems". In: *Communications of the ACM* 21.2 (1978), pp. 120–126.

[Sha49] Claude E Shannon. "Communication theory of secrecy systems". In: *The Bell system technical journal* 28.4 (1949), pp. 656–715.

[Wik25a] Wikipedia. *Enigma machine — Wikipedia, The Free Encyclopedia*. `http://en.wikipedia.org/w/index.php?title=Enigma%20machine&oldid=1318813039`. [Online; accessed 04-November-2025]. 2025.

[Wik25b] Wikipedia. *Gene Grabeel — Wikipedia, The Free Encyclopedia*. `http://en.wikipedia.org/w/index.php?title=Gene%20Grabeel&oldid=1245098220`. [Online; accessed 04-November-2025]. 2025.

[Wik25c] Wikipedia. *History of cryptography — Wikipedia, The Free Encyclopedia*. `http://en.wikipedia.org/w/index.php?title=History%20of%20cryptography&oldid=1319805209`. [Online; accessed 04-November-2025]. 2025.

[Wik25d] Wikipedia. *Mary, Queen of Scots — Wikipedia, The Free Encyclopedia*. http://en.wikipedia.org/w/index.php?title=Mary%2C%20Queen%20of%20Scots&oldid=1319768164. [Online; accessed 04-November-2025]. 2025.